

WET2VO Web Technologies

Vorlesungsunterlagen

Wolfgang Hochleitner
wolfgang.hochleitner@fh-hagenberg.at

SS 2026

Vorlesung 7

Formularverarbeitung

7.1 Wiederholung: Formulardatenverarbeitung

Die *Verarbeitung von Formulardaten* wurde bereits in Abschnitt 3.5 grundsätzlich besprochen. Hier wird nochmals eine kurze Wiederholung gegeben, bevor noch einige weitere Konzepte besprochen werden, die im Kapitel 3 noch ausgelassen wurden, jedoch für eine sichere und Verarbeitung von Daten unbedingt erforderlich sind.

7.1.1 Das HTML-Formular

Zur Formularübertragung wird ein HTML-Formular, bestehend aus dem `<form>`-Element, mindestens einem Formularelement (z. B. `<input>`) und meist einem Button zum Absenden (z. B. `<button>`) benötigt. Ein einfaches Formular sieht etwa wie folgt aus:

```
1 <form method="post" action="<?= $_SERVER["SCRIPT_NAME"] ?>">
2   <label for="datenlabel">Info:</label>
3   <input type="text" name="daten" id="datenlabel">
4   <button type="submit">Abschicken</button>
5 </form>
```

Dieses Formular verwendet die POST-Übertragungsmethode (siehe dazu Abschnitt 7.1.2) im `action`-Attribut und ruft sich selbst wieder auf. Dies wird durch das Einsetzen von `$_SERVER["SCRIPT_NAME"]` erreicht. Dieser Eintrag im superglobalen Array `$_SERVER` enthält den Pfad zum aktuellen Skript. Somit kann dieses frei umbenannt werden, ohne dass der Inhalt darauf angepasst werden muss.

7.1.2 Übertragungsmethode und Zugriff

Formulare können in PHP mit den HTTP Methoden GET oder POST übertragen werden. Angegeben wird dies im `action`-Attribut des Formulars mit den Werten `get` oder `post`. Der Unterschied liegt in der Übertragung der Inhalte.

Übertragung mittels GET

GET-Parameter werden in einem sogenannten Query-String an den URL gehängt. Dies geschieht im Format `?key1=value1&key2=value2`.

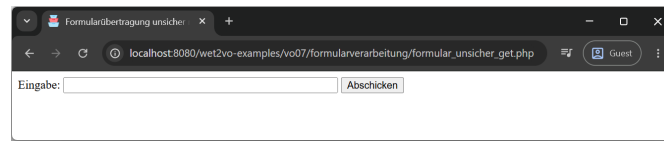


Abbildung 7.1: Das Formular zeigt ein einfaches Formularfeld zum Eingeben eines Wertes.

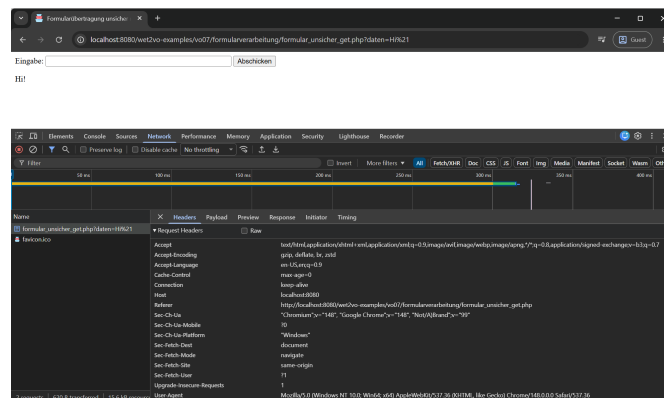


Abbildung 7.2: Die Request-Header für eine Formularübertragung mit GET. Es ist ersichtlich, dass keine Daten übertragen werden, diese hängen ja am URL.

Wird das obige Formular in der Datei `formular_unsicher_get.php` gespeichert, sein `action`-Attribut auf `get` geändert, und über `http://localhost:8080/wet2vo-examples/vo07/formularverarbeitung/formular_unsicher_get.php` aufgerufen, so wird ein Formular wie in Abbildung 7.1 gezeigt.

Wird in das Feld mit dem Namen `daten` (definiert über das `name`-Attribut im Formular) „Hi!“ eingegeben, erfolgt ein GET-Request an den Server, der wie folgt aussieht: `http://localhost:8080/wet2vo-examples/vo07/formularverarbeitung/formular_unsicher_get.php?daten=Hi%21`. Es werden also der Schlüssel „daten“ und der Wert „Hi!“ im Format `?daten=Hi%21` an den ursprünglichen URL gehängt. Dieser ist derselbe, weil im `action`-Attribut angegeben wurde, dass sich die Datei selbst aufrufen soll. `%21` ist hier das Ausrufezeichen, welches URL-kodiert wurde, da in URLs nur eine bestimmte Menge von erlaubten Zeichen vorkommen dürfen.

Der Request kann in den DevTools des Browsers im Tab `Network > Headers` betrachtet werden. Er ist in Abbildung 7.2 für den vorigen Request dargestellt.

Betrachtet man die Payload, also die Daten, die mitübertragen werden – verfügbar in den DevTools über `Network > Payload`, so ist in Abbildung 7.3 ersichtlich, dass hier von „Query String Parameters“ gesprochen wird. Bei einem genaueren Blick auf die Request Header ist auch ersichtlich, dass nirgends eine Größenangabe in Bytes mitgeschickt wird, da der Request eben keinerlei Daten enthält. Bei einer POST-Übertragung ist dies anders (siehe nächster Abschnitt).

Der Zugriff erfolgt über das superglobale Array `$_GET` an der Stelle des Formularfeldnamens, also in diesem Fall über `$_GET["daten"]`. Um den eingegebenen Wert

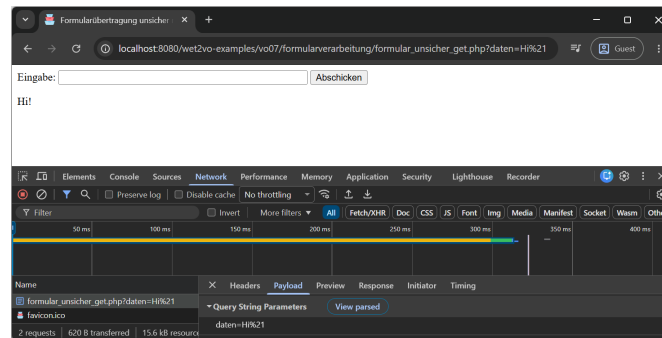


Abbildung 7.3: Die Payload, also die mitgeschickten Daten bei einer GET-Übertragung, ist nur als Query String Parameter ausgewiesen.

unterhalb des Formularfeldes anzuzeigen (wie in den Abbildungen 7.2 und 7.3 ersichtlich), ist etwa folgender Code nötig:

```
1 if (isset($_GET["daten"])) {
2     echo "<p>".$_GET["daten"]."</p>";
3 }
```

Die Abfrage mit `isset()` stellt sicher, dass nur dann eine Ausgabe erfolgt, wenn `$_GET["daten"]` auch vorhanden ist. Dies ist nur nach dem Absenden des Formulars der Fall. Diese Art der Ausgabe ist jedoch unsicher. Dies wird in Abschnitt 7.2.1 erläutert. In Abschnitt 7.2.5 werden Gegenstrategien vorgestellt.

Übertragung mittels POST

POST-Parameter werden gesondert im Request-Body als Schlüssel/Wert-Paare übertragen und tauchen im URL nicht auf.

Wird der Code für das Formular in eine Datei `formular_unsicher_post.php` gespeichert, und über `http://localhost:8080/wet2vo-examples/vo07/formularverarbeitung/formular_unsicher_post.php` aufgerufen, so sieht das Ergebnis zunächst identisch zu dem in Abbildung 7.1 aus.

Wird in das Feld mit dem Namen `daten` wieder „Hi!“ eingegeben, erfolgt nun ein *POST*-Request an den Server, der wie folgt aussieht: `http://localhost:8080/wet2vo-examples/vo07/formularverarbeitung/formular_unsicher_post.php`. Es handelt sich also wieder um denselben URL wie zuvor, Parameter sind keine sichtbar. Diese werden im Request-Body gesondert übertragen. Dies kann wieder in den DevTools über **Network** » **Headers** betrachtet werden. Der Request ist in Abbildung 7.4 für den vorigen Request dargestellt.

Beim Blick auf die Payload des Requests (**Network** » **Payload**) zeigt sich in Abbildung 7.5, dass hier wirklich Daten verschickt werden (die Daten werden nun als „Form Data“ bezeichnet). Diese sind in der Form `key=value` hinterlegt und lauten in diesem Fall `daten=Hi%21`. Zählt man die Zeichen, die jeweils 1 Byte benötigen, so kommt man auf die in der Header-Angabe ausgewiesenen 11 Bytes. Diese Form der Übertragung ist im Content-Type `application/x-www-form-urlencoded` festgelegt und stellt die Standardübertragung für Formulare mit reinem Textinhalt dar. Der Content-Type ist dabei auch in den Request-Headern (Abbildung 7.4) ersichtlich. Er muss nicht speziell

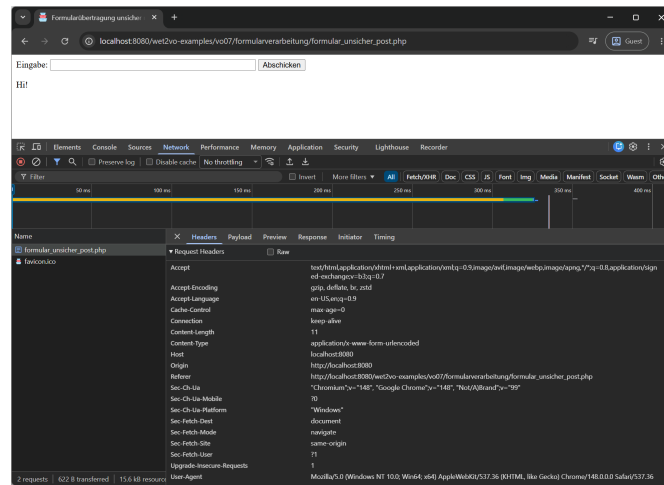


Abbildung 7.4: Die Request-Header für eine Formularübertragung mit POST. Der Eintrag **Content-Length** zeigt an, dass hier tatsächlich Inhalt im Request mitgesendet wird. In diesem Fall 11 Bytes. Der **Content-Type** der Übertragung lautet **application/x-www-form-urlencoded**.

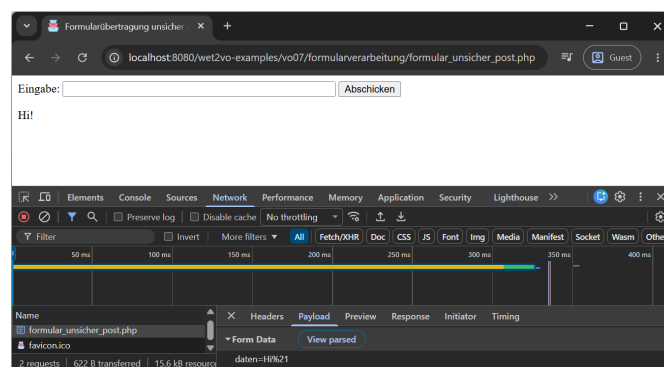


Abbildung 7.5: Die Payload für eine POST Übertragung ist ein URL-kodierter String aller Parameter. Der Browser deutet dies über „Form Data“ an.

angegeben werden.

Der Zugriff erfolgt über das superglobale Array `$_POST` an der Stelle des Formularfeldnamens, also in diesem Fall über `$_POST["daten"]`. Um den eingegebenen Wert unterhalb des Formularfeldes anzuzeigen (wie in den Abbildungen 7.4 und 7.5 ersichtlich), ist etwa folgender Code nötig:

```
1 if (isset($_POST["daten"])) {
2     echo "<p>{"$_POST["daten"]}</p>";
3 }
```

Auch dieser Zugriff weist ein schwerwiegendes Sicherheitsproblem auf, welches in Abschnitt 7.2.1 besprochen wird.

Vergleich GET und POST

GET- und POST-Übertragungen bieten beide Vor- und Nachteile, die im Folgenden aufgelistet werden. Der Vorteil einer Methode ist meist im Umkehrschluss der Nachteil der anderen.

Vorteile von GET-Übertragungen:

- Der Request kann mit allen Parametern als Lesezeichen (Bookmark) gespeichert oder geteilt werden.
- Leichteres Debuggen, da die übermittelten Eingaben im URL sichtbar und auch veränderbar sind.
- Die Zielseiten (URLs mit Parametern) können von Suchmaschinen indiziert werden.

Vorteile von POST-Übertragungen:

- Die übermittelten Formulardaten sind nicht im Verlauf (History) des Browsers enthalten. Dies bedeutet mehr Sicherheit.
- Da die Parameter nicht im URL übertragen werden, gibt es keine Längen- bzw. Mengenbeschränkung der Daten (bei GET durch die URL-Maximallänge von 2048 Zeichen der Fall).
- Mit POST können Dateien versandt werden (siehe Abschnitt 7.3).
- POST-Requests werden vom Browser nicht zwischengespeichert (cached).

Generell ist meist POST zu bevorzugen, wenn ein Formular mit Daten aktiv verschickt wird. GET macht dann Sinn, wenn die Parameter Dinge enthalten, die ein bestimmtes Verhalten nachstellen sollen, etwa die Suche eines bestimmten Produkts in einem Webshop: <https://somewebshop.site/search?query=someproduct>.

7.2 Gefahren für Webanwendungen

Web-Anwendungen, die mit Eingaben von Benutzer*innen arbeiten, sind bei der Weiterverarbeitung solcher Eingaben prinzipiell einigen Gefahren ausgesetzt. Diese müssen erkannt und entsprechend neutralisiert werden. Im Folgenden wird ein kurzer Überblick über mögliche Probleme und Gefahren und deren Lösungen gegeben.

7.2.1 Cross-Site-Scripting

Cross-Site-Scripting, kurz XSS, bezeichnet eine Sicherheitslücke (bzw. deren Ausnutzung), die durch das ungeprüfte Weiterverwenden von Benutzer*inneneingaben entsteht. Ziel des Cross-Site-Scriptings kann im schlimmsten Fall das Ausspionieren von sensiblen Daten der User*innen sein.

Bei XSS handelt es sich um eine Art der so genannten *HTML-Injection*, also das ungewollte Einfügen von HTML-Code zwischen zwei Aufrufen einer Seite. Dieser HTML-Code ist meist schadhaftes JavaScript, daher auch der Ausdruck Cross-Site-Scripting.

XSS kann immer dann auftreten, wenn Eingaben von Benutzer*innen ungeprüft wieder ausgegeben werden. Angenommen, es gibt, wie in Abschnitt 7.1 gezeigt, ein Eingabefeld mit dem Namen `daten` und dessen Inhalt wird beim Absenden so ausgegeben:

```
1 echo "<p>{$_POST["daten"]}</p>";
```

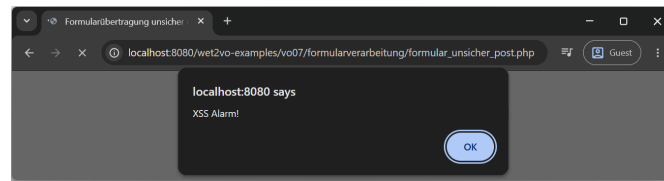


Abbildung 7.6: Die ungeprüfte Eingabe eines HTML-Fragments mit JavaScript lässt die Alert-Box im Browser erscheinen.

Dies reicht aus, um schadhafte Skripte ausführen zu können. Die folgende Eingabe im Formularfeld erzeugt bereits den ungewollten Aufruf eines Skripts (siehe Abbildung 7.6):

```
1 <script>alert("XSS Alarm!")</script>
```

Da das PHP-Skript einfach den Inhalt des Formularfeldes ausgibt, wird auch die Eingabe in Form eines HTML-`<script>`-Elements genauso bei der Ausgabe erzeugt. In diesem Fall wird nur ein Popup geöffnet, es könnte jedoch weit mehr mittels JavaScript aufgerufen werden, etwa

- das unendliche Neuladen der Seite mittels `location.reload()`,
- die Umleitung von Benutzer*innen über den Aufruf von JavaScript-Code wie `location.href = "https://malicious.site"`,
- das Auslesen von Cookies der aktuellen Seite über `document.cookie` (sofern diese nicht durch das `HttpOnly`-Flag geschützt sind),
- das Auslesen von `localStorage` und `sessionStorage`, wo moderne Webapplikationen häufig sensible Zugriffs-Tokens (wie JWTs) ablegen,
- das Auslesen aller gedrückten Tasten über Key-Events (Keylogging) und das Versenden derselben an eine Zieladresse,
- das Verändern der angezeigten Webseite (Defacement) oder das dynamische Einblenden von gefälschten Formularfeldern (Phishing) im visuellen Kontext der eigentlich vertrauenswürdigen Domain,
- das unbemerkte Ausführen von Aktionen im Namen des Opfers (etwa das Ändern von Passwörtern, Hinzufügen von Admin-Accounts oder Tätigen von Käufen) via `fetch()`-Requests, da das Skript mit der aktiven Sitzung der eingeloggten Person läuft,
- und viele weitere Dinge, die meist noch viel unerfreulicher sind (wie etwa das Ausführen von Krypto-Minern im Hintergrund).

7.2.2 SQL-Injection

Die *SQL-Injection* basiert auf einem ähnlichen Konzept. Speichert eine Seite Eingaben seiner Benutzer*innen in einer SQL-Datenbank und der Input besteht aus SQL-Anweisungen, so können diese ungefiltert in die Datenbank gelangen und dort auf diese wirken. So können etwa Datensätze verändert oder ausgespäht werden.

Ein Beispiel wäre eine Suche, z. B. in einem Webshop. Angenommen, es gibt eine Seite `find.php`, die einen GET-Parameter `id` verwendet, um ein Produkt mit einer

bestimmten ID-Nummer zu suchen. Diese könnte wie folgt aufgerufen werden: <https://localhost/mysite/find.php?id=42>

Das PHP-Skript verwendet dann den Wert des GET-Parameters (hier 42) ungefiltert in einer SQL-Abfrage weiter:

```
1 SELECT author, subject, text FROM artikel WHERE ID=42
```

Angreifer*innen können sich dies zunutze machen, indem der Wert des GET-Parameters um ein Update-Statement erweitert wird: [https://localhost/mysite/find.php?id=42; UPDATE+USER+SET+TYPE="admin"+WHERE+ID=23](https://localhost/mysite/find.php?id=42; UPDATE+USER+SET+TYPE='admin'+WHERE+ID=23)

Somit wird an die ursprünglichen SQL-Abfrage noch eine zweite angehängt, in diesem Fall wird der*die User*in mit der ID 23 zum*zur Administrator*in gemacht. Wenn dies der*die User*in der angreifenden Person ist, besteht damit umfangreicher Zugriff auf das System:

```
1 SELECT author, subject, text FROM artikel WHERE ID=42; UPDATE USER SET TYPE="admin"
WHERE ID=23
```

Das Thema SQL-Injection wird detaillierter in der Lehrveranstaltung Relational Databases (RDB2VO) behandelt.

7.2.3 Cross-Site Request Forgery (CSRF)

Bei der *Cross-Site Request Forgery*, kurz CSRF, führen fremde Benutzer*innen auf einer Webseite einen Request mit den Rechten von eingeloggten User*innen ohne deren Wissen aus. Wenn diese Opfer über administrativen Zugang verfügen, kann der Schaden das gesamte System betreffen.

Dazu wird meistens ein Link präpariert, der vom System als gültige Anweisung interpretiert wird. Weiß eine angreifende Person etwa, dass über den Link <https://localhost/mysite/user.php?action=new&user=me&pass=secret> ein neuer Account „me“ angelegt werden kann, muss sie es nur noch schaffen, diesen Link einer administrativen Kraft der Seite unterzububeln (z. B. versteckt in einer E-Mail oder auf einer präparierten Webseite). Ist diese Person in ihrem Browser noch mit erweiterten Rechten angemeldet und klickt auf den Link, hängt der Browser das gültige Session-Cookie automatisch an den Request an – der neue Account wird angelegt. Der*die Angreifer*in kann das System anschließend übernehmen.

Wichtig: Das obige Beispiel verwendet einen GET-Request, die bloße Verwendung von POST-Requests für Zustandsänderungen schützt jedoch nicht vor CSRF! Angreifende können auf ihrer präparierten Webseite versteckte HTML-Formulare platzieren, die beim Laden der Seite via JavaScript automatisch und unbemerkt per POST an die Zielseite abgesendet werden. Auch HTTPS bietet hier *keinen* Schutz, da die Authentifizierung (das Cookie) vom Browser bei jedem Request automatisch mitgesendet wird, unabhängig davon, wer den Request initiiert hat.

Als wirksame Gegenmaßnahmen sind in der modernen Webentwicklung folgende Konzepte essenziell:

- **SameSite-Cookies:** Dies ist die wichtigste moderne Verteidigungslinie. Wird beim Setzen des Session-Cookies entweder das Attribut **SameSite=Lax** oder auch **SameSite=Strict** verwendet, verbietet der Browser das Mitsenden des Cookies bei Anfragen, die von einer fremden Domain (Cross-Site) stammen.

- **Anti-CSRF-Tokens:** Hierbei handelt es sich um das *Synchronizer Token Pattern*. Bei jedem Formularaufruf generiert der Server eine lange, kryptografisch sichere Zufallszeichenkette (Token) und speichert diese in der Session. Dasselbe Token wird als verstecktes Feld in das HTML-Formular eingefügt. Beim Absenden prüft der Server, ob das mitgesendete Token mit dem in der Session übereinstimmt. Angreifer*innen können dieses Token nicht erraten.
- **Re-Authentifizierung:** Bei besonders kritischen Aktionen (Passwortänderung, Account-Löschung, Überweisungen) sollte immer das aktuelle Passwort erneut abgefragt werden.
- **XSS-Prävention:** Eine Cross-Site-Scripting-Schwachstelle (siehe Abschnitt 7.2.1) hebt jeden CSRF-Schutz aus. Werden Skripte im Kontext der Zielseite ausgeführt, können diese Tokens auslesen und in beliebigen Requests mitsenden. Ein stringenter XSS-Schutz ist daher zwingend erforderlich.

Das Grundproblem von CSRF ist untrennbar mit dem Status des Logins verbunden, weshalb Sessions in Vorlesung 8 im Detail thematisiert werden.

7.2.4 OWASP Top Ten

OWASP, kurz für *Open Web Application Security Project*, ist eine Non-Profit-Organisation, die sich das Ziel gesetzt hat, die Sicherheit von Webapplikationen zu verbessern. Unter <https://owasp.org/www-project-top-ten/> gibt die OWASP in mehrjährigen Abständen die Top 10 der kritischsten Sicherheitsprobleme heraus. Dort werden die Probleme im Detail erläutert und Gegenstrategien vorgeschlagen.

Die aktuell gültige Liste stammt aus dem Jahr 2025 und umfasst die folgenden Top 10:

- A01: Broken Access Control,
- A02: Security Misconfiguration,
- A03: Software Supply Chain Failures,
- A04: Cryptographic Failures,
- A05: Injection,
- A06: Insecure Design,
- A07: Authentication Failures,
- A08: Software or Data Integrity Failures,
- A09: Security Logging and Alerting Failures,
- A10: Mishandling of Exceptional Conditions.

Die Punkte Cross-Site-Scripting und SQL-Injection sind dabei weiterhin unter „Injection“ (A05) zusammengefasst und gehören zu den kritischsten Problemen von Webapplikationen. Cross-Site Request Forgery (CSRF) ist kein eigenständiger Punkt, sondern ist in „Broken Access Control“ (A01) integriert, da es im Kern eine missbräuchliche Rechteauserweiterung darstellt.

Konzepte zur Vermeidung von Problemen bei „Broken Access Control“ (A01) und „Authentication Failures“ (A07) werden in Vorlesung 8 detaillierter behandelt. Der Punkt „Software Supply Chain Failures“ (A03) betrifft in PHP-Projekten vor allem über Composer installierte Fremd-Komponenten – eine kompromittierte Abhängigkeit

(Dependency) kann das gesamte Projekt gefährden. Eine regelmäßige Überprüfung und Aktualisierung ist hier zwingend erforderlich.

7.2.5 PHP Filter-Funktionen

Wie in Abschnitt 7.2.1 bereits erwähnt wurde, ist die ungeprüfte Ausgabe von Benutzer*inneneingaben die Hauptursache für Cross-Site-Scripting. Als Devise muss daher gelten:

Keine Benutzer*inneneingaben dürfen in PHP ungefiltert (d. h. ohne entsprechende Überprüfung) wieder ausgegeben werden!

Für eine derartige Überprüfung, auch *Filterung* genannt, stellt PHP einige Funktionen zur Verfügung:

htmlspecialchars

`htmlspecialchars($string [, $flags [, $encoding]])`¹ wandelt fünf der wichtigsten, in HTML reservierten, Zeichen in ihre Entitäten um:

- `&` – wird zu `&`;
- `<` – wird zu `<`;
- `>` – wird zu `>`;
- `"` – wird zu `"`; (je nach gesetzten Flags),
- `'` – wird zu `'` oder `'`; (je nach gesetzten Flags).

Der Parameter `$flags` erlaubt mehrere Optionen, die mit dem bitweisen ODER (`|`) verknüpft werden können. Die wichtigsten sind:

- `ENT_COMPAT`: Konvertiert nur doppelte Anführungszeichen zu `"`; und lässt einfache Anführungszeichen unverändert.
- `ENT_QUOTES`: Konvertiert sowohl doppelte (`"`;) als auch einfache Anführungszeichen (`'` bei HTML5, sonst `'`).
- `ENT_NOQUOTES`: Lässt doppelte und einfache Anführungszeichen unverändert.
- `ENT_SUBSTITUTE`: Ersetzt ungültige Code-Unit-Sequenzen mit dem Unicode-Ersatzzeichen U+FFFD (eine Raute mit einem Fragezeichen).
- `ENT_HTML401`: Handle Code als HTML 4.01.
- `ENT_HTML5`: Handle Code als HTML5.

`ENT_QUOTES | ENT_SUBSTITUTE | ENT_HTML401` ist der Standardwert für `$flags` seit PHP 8.1. Wird PHP Version 5.4 oder höher verwendet, sollte der letzte Teil auf HTML5 angepasst werden, vorausgesetzt HTML5 kommt im Dokument zum Einsatz.

`$encoding` erlaubt die Angabe des verwendeten Zeichensatzes. Der Standardwert ist seit PHP 5.6 der aktuell konfigurierte Standardzeichensatz (in der Regel UTF-8) und muss nicht mehr angegeben werden. Bei älteren PHP-Versionen ist dies noch nötig.

¹<https://www.php.net/manual/de/function htmlspecialchars.php>

htmlspecialchars

`htmlspecialchars()`² hat dieselben Parameter wie `htmlspecialchars()`. Die Funktion ist auch ansonsten komplett identisch, wandelt jedoch *alle* Zeichen, für die es HTML-Entitäten gibt, in solche um. Also auch etwa „ä“ zu `ä`, „ö“ zu `ö`; usw.

filter_var und filter_input

Universeller sind die beiden Filterfunktionen `filter_var($variable [, $filter])`³ und `filter_input($type, $variable_name [, $filter])`⁴. Erstere filtert den Inhalt einer Variable, durch Angabe des Filters `FILTER_SANITIZE_FULL_SPECIAL_CHARS` wird das Verhalten der Funktion `htmlspecialchars()` erreicht. Dieser Filter kann auch bei `filter_input()` verwendet werden. Bei dieser Funktion wird nicht der Inhalt einer Variable, sondern der Inhalt eines `$_GET`- oder `$_POST`-Eintrages gefiltert. Die folgenden zwei Zeilen machen prinzipiell dasselbe (lediglich bei den Rückgabewerten gibt es kleine Unterschiede):

```
1 filter_input(INPUT_POST, "daten", FILTER_SANITIZE_FULL_SPECIAL_CHARS);
2 htmlspecialchars($_POST["daten"], ENT_QUOTES | ENT_HTML401);
```

strip_tags

Die Funktion `strip_tags($string [, $allowable_tags])`⁵ entfernt alle HTML- und PHP-Tags aus einem String. Mit dem optionalen Parameter `$allowable_tags` können alle Tags angegeben werden, die nicht entfernt werden sollen.

Diese Funktion eignet sich aufgrund einiger Umgehungsmöglichkeiten kaum für einen effektiven XSS-Schutz und sollte nicht eingesetzt werden. So wird beispielsweise der Tag `<a href="javascript:alert(1)">` von `strip_tags` nicht bereinigt, wenn `<a>` erlaubt ist.

Beispiel

Im folgenden Beispiel wird der Text `<h1>"Häullo" 'World'</h1>` in ein Formularfeld eingegeben. Die Ausgabe erfolgt in der ersten Zeile ungefiltert (also potentiell gefährlich), dann mittels `htmlspecialchars()`, `htmlspecialchars()`, `filter_var()`, `filter_input()` und schließlich `strip_tags()`. Das Ergebnis ist in Abbildung 7.7 zu sehen, der generierte Quellcode (zwischen den `<div>`-Elementen) sieht wie folgt aus:

```
1 unfiltered:      <h1>"Häullo" 'World'</h1>
2 htmlspecialchars: &lt;h1&gt;&quot;Häullo&quot; &apos;World&apos;&lt;/h1&gt;
3 htmlspecialchars: &lt;h1&gt;&quot;H&auml;llo&quot; &apos;World&apos;&lt;/h1&gt;
4 filter_var:      &lt;h1&gt;&quot;H&auml;llo&quot; &#039;World&#039;&lt;/h1&gt;
5 filter_input:    &lt;h1&gt;&quot;H&auml;llo&quot; &#039;World&#039;&lt;/h1&gt;
6 strip_tags:      "Häullo" 'World'
```

²<https://www.php.net/manual/de/function.htmlspecialchars.php>

³<https://www.php.net/manual/de/function.filter-var.php>

⁴<https://www.php.net/manual/de/function.filter-input.php>

⁵<https://www.php.net/manual/de/function.strip-tags.php>



Abbildung 7.7: Ungefiltert wird die Eingabe `<h1>"Hällo" 'World'</h1>` tatsächlich als Überschrift erster Ordnung angezeigt. Mit den Filter-Funktionen werden die HTML-Spezialzeichen in Entitäten umgewandelt, sodass sie nur mehr als Zeichen erscheinen, oder ganz entfernt.

7.2.6 Symfony HTML Sanitizer: Komponente zur Filterung von Inhalten

Zur Verarbeitung von User*innendaten aus Formularen ist `htmlspecialchars()` eine verlässliche und meist absolut ausreichende Methode. Es gibt jedoch auch Fälle, in denen formatierte HTML-Eingaben erlaubt und sicher gefiltert werden müssen. Ein typisches Beispiel sind Eingaben aus visuellen Editoren (WYSIWYG) in Content-Management-Systemen. Hier soll Textformatierung (z. B. `<b>`, `<i>`) erhalten bleiben, während Cross-Site-Scripting-Vektoren (z. B. `<script>`-Tags oder `onload`-Attribute) zuverlässig entfernt werden müssen.

Die moderne Standardkomponente hierfür ist `symfony/html-sanitizer` aus dem Symfony-Framework. Er wird über Composer installiert:

```
1 $ composer require symfony/html-sanitizer
```

Die Nutzung basiert auf einem strikten „Allowlisting“-Ansatz. Es wird zunächst eine Konfiguration erstellt, die definiert, was erlaubt ist. Die Methode `allowSafeElements()` ist dabei ein guter Startpunkt, da sie Standard-Tags für Textauszeichnungen zulässt, aber jegliche ausführende Logik blockiert. Anschließend wird das eigentliche Sanitizer-Objekt erzeugt:

```
1 use Symfony\Component\HtmlSanitizer\HtmlSanitizerConfig;
2 use Symfony\Component\HtmlSanitizer\HtmlSanitizer;
3
4 $config = new HtmlSanitizerConfig()->allowSafeElements();
5 $sanitizer = new HtmlSanitizer($config);
```

Nun kann das Formularfeld (in diesem Fall `daten`) über die Methode `sanitize()` bereinigt werden, bevor es ausgegeben oder in einer Datenbank gespeichert wird:

```
1 if (isset($_POST["daten"])) {
2     $datenBereinigt = $sanitizer->sanitize($_POST["daten"]);
3     echo "<div>$datenBereinigt</div>";
4 }
```

Die Ausgabe gestaltet sich prinzipiell wie folgt:

- Sicheres HTML (wie `<p>`, `<strong>`) bleibt erhalten.
- Schadhaftes HTML (z. B. `<script>`, `<iframe>` oder `javascript:-`Links) wird restlos entfernt.

- Fehlerhaft verschachteltes HTML wird vom Sanitizer automatisch erkannt und strukturell korrigiert, um eine valide Ausgabe (W3C-Standard) zu gewährleisten.

Aufgrund der enormen Komplexität von HTML und den sich ständig wandelnden Angriffsvektoren sollte für das Filtern von HTML *immer* eine spezialisierte, gut gewartete Komponente wie der Symfony HTML Sanitizer herangezogen und niemals versucht werden, dies mit regulären Ausdrücken selbst zu erstellen.

7.3 Dateiuploads

Das *Hochladen von Dateien* gehört – wie auch das Erfassen von textuellen Eingaben der Benutzer*innen – zum Themengebiet der Formularverarbeitung. Während sich jedoch textuelle Daten rein clientseitig erfassen und dann z. B. per E-Mail versenden lassen, können hochgeladene Dateien nur serverseitig weiterverarbeitet werden. Dafür sind zunächst einige Dinge notwendig.

7.3.1 Voraussetzungen und Ablauf

Für das Erfassen von Dateien sind einige Voraussetzungen zu beachten. Zunächst muss ein entsprechendes Formular vorhanden sein, aber auch die maximal erlaubte Dateigröße spielt eine Rolle.

Formular

Beim Formular für einen Dateiupload spielen zwei Attribute eine besondere Rolle. Zunächst muss beim `<form>`-Element selbst das Attribut `enctype` mit dem speziellen Wert `multipart/form-data` angegeben sein, um die Übertragung größerer Datenmengen zu ermöglichen. Des Weiteren muss es ein `<input>`-Element geben, dessen `type`-Attribut den Wert `file` aufweist. Dies erzeugt ein Eingabefeld mit dem typischen „Browse“-Button, der es ermöglicht, eine Datei im Dateisystem für den Upload auszuwählen.

Anmerkung: Das Aussehen des Dateiupload-Eingabefeldes wird sowohl vom verwendeten Betriebssystem als auch vom jeweiligen Browser bestimmt. Sein Aussehen variiert daher stark. Auch eine Formatierung mittels CSS ist nur mit einigen Tricks möglich.

Weiters ist bei Dateiuploads die Übertragungsmethode POST vorausgesetzt.

Wie bereits eingehend erwähnt, ist es für derartige Formulare üblich, eine Größenbeschränkung für Dateien anzugeben. Dies kann auf zwei Arten geschehen:

- mittels verstecktem Formularfeld `MAX_FILE_SIZE` (Soft-Limit),
- durch Einstellungen in der Konfigurationsdatei `php.ini` (Hard-Limit).

Die Angabe über ein verstecktes Formularfeld (`type="hidden"`) muss *vor* dem Eingabefeld für Dateien erfolgen. Das Feld muss (durch das Attribut `name`) den Namen `MAX_FILE_SIZE` haben, das Attribut `value` enthält die Obergrenze in Bytes. Die Angabe einer Obergrenze auf diese Art und Weise ist keineswegs sicher und kann clientseitig leicht ausgehebelt werden (sie steht ja im Quelltext). Trotzdem kann es sinnvoll sein, sie anzugeben, da der Browser *sofort* weiß, wie groß die Datei maximal sein darf, noch bevor er mit dem Upload begonnen hat. Es verhindert also, dass Benutzer*innen auf den Upload einer großen Datei warten, nur um dann die Fehlermeldung zu erhalten,

dass die Datei zu groß war. Derzeit implementiert jedoch kein moderner Browser eine derartige Überprüfung, weshalb sie auch durchaus ignoriert werden kann.

Ein fertiges HTML-Formular inklusive maximaler Uploadgröße sieht wie folgt aus:

```
1 <form method="post" enctype="multipart/form-data"
   action="<?= $_SERVER["SCRIPT_NAME"] ?>">
2   <input type="hidden" name="MAX_FILE_SIZE" value="2097152">
3   <input type="file" name="userfile">
4   <button type="submit">Hochladen</button>
5 </form>
```

Die angegebene Maximalgröße von 2.097.152 Bytes entspricht genau 2 Megabytes ($1024 \cdot 1024 \cdot 2$ Bytes).

php.ini Einstellungen

Wichtiger als die Angabe eines Soft-Limits über das Formular, ist das so genannte Hard-Limit in der Konfigurationsdatei `php.ini`. Dieses kann von den Nutzer*innen (im Gegensatz zur Angabe im Formular) nicht umgangen werden. Zusätzlich können hier weitere Dinge rund um Dateiuploads gesteuert werden.

Tabelle 7.1 enthält eine Übersicht über alle Konfigurationsvariablen in der `php.ini`, die einen Einfluss auf das Hochladen von Dateien haben.

Je nach Webserver-Konfiguration können andere Werte für die einzelnen Variablen voreingestellt sein. Die Werte können jedoch einfach geändert werden. Dabei ist zu beachten, dass nach einer Änderung ein Neustart des Apache Servers nötig ist, um die Konfigurationsdatei neu einzulesen.

7.3.2 Verarbeitung im superglobalen Array

Nachdem das Formularfeld sowie der Webserver auf Dateiuploads vorbereitet wurden, kann mit der Verarbeitung begonnen werden. Nachdem das Formular abgeschickt wurde, wird die Datei in ein temporäres Verzeichnis auf den Webserver hochgeladen (dieses kann in `php.ini` im Eintrag `upload_tmp_dir` festgelegt werden). Um das Überschreiben von eventuell gleichlautenden, bereits hochgeladenen Dateien zu vermeiden, bekommt die Datei zunächst einen zufällig gewählten Namen.

Der Zugriff auf die hochgeladene Datei erfolgt in weiterer Folge über ein weiteres superglobales Array: `$_FILES`.

Das Array ist zweidimensional und hat sechs Eigenschaften, die Auskunft über die hochgeladene Datei geben. Im Folgenden wird angenommen, dass der Name des Upload-Feldes `userfile` lautet:

- `$_FILES["userfile"]["name"]`: Der ursprüngliche Name der Datei, wie er auf dem Computer des*der Benutzer*in lautete.
- `$_FILES["userfile"]["full_path"]`: Der vollständige Pfad, wie er vom Browser übermittelt wurde (seit PHP 8.1).
- `$_FILES["userfile"]["type"]`: Der MIME-Type der hochgeladenen Datei. Bei einem GIF-Bild etwa `image/gif`, Bei einer Textdatei lautet der Wert `text/plain`.
- `$_FILES["userfile"]["size"]`: Die Größe der hochgeladenen Datei in Bytes.
- `$_FILES["userfile"]["tmp_name"]`: Der temporäre Dateiname, unter dem die hochgeladene Datei auf dem Server gespeichert wurde.

Tabelle 7.1: Die Konfigurationsvariablen aus `php.ini`, die das Hochladen von Dateien beeinflussen. Die Werte in der zweiten Spalte sind Beispiele – andere bzw. ähnliche Werte sind möglich.

Variable	Wert	Bedeutung
<code>upload_max_filesize</code>	2M	Maximale Größe, die eine hochgeladene Datei haben darf.
<code>file_uploads</code>	On	Legt fest, ob generell Dateiuploads per HTTP erlaubt sind.
<code>post_max_size</code>	8M	Setzt die maximal erlaubte Größe von POST-Daten.
<code>max_file_uploads</code>	20	Die maximale Anzahl an Dateien, die in einem Request hochgeladen werden können.
<code>memory_limit</code>	128M	Setzt den Maximalwert des Speichers, den ein Skript verbrauchen darf. –1 legt fest, dass es kein Limit gibt.
<code>max_execution_time</code>	30	Legt die maximale Zeit in Sekunden fest, die ein Skript laufen darf, bevor der Parser die Ausführung stoppt.
<code>max_input_time</code>	60	Legt die maximale Zeit in Sekunden fest, die ein Skript verbrauchen darf, um Eingabedaten (wie POST, GET und Dateiuploads) zu verarbeiten.

- `$_FILES["userfile"]["error"]`: Der Fehlercode, der im Zusammenhang mit dem Hochladen der Datei generiert wurde.

7.3.3 Fehlercodes

Nachdem die Datei hochgeladen wurde, wird die Eigenschaft `error` des superglobalen Arrays `$_FILES` immer mit einem Wert befüllt. Dieser Wert ist ein Integer, für den es jedoch sprechendere Konstanten gibt:

- `UPLOAD_ERR_OK` (0): Kein Fehler, die Datei wurde erfolgreich hochgeladen.
- `UPLOAD_ERR_INI_SIZE` (1): Die Größe der hochgeladenen Datei überschritt den in der `php.ini` unter `upload_max_filesize` eingestellten Wert.
- `UPLOAD_ERR_FORM_SIZE` (2): Die Größe der Datei überschritt den im Formular unter `MAX_FILE_SIZE` angegebenen Wert.
- `UPLOAD_ERR_PARTIAL` (3): Die Datei wurde nur zum Teil hochgeladen.
- `UPLOAD_ERR_NO_FILE` (4): Es wurde keine Datei hochgeladen.
- `UPLOAD_ERR_NO_TMP_DIR` (6): Es existiert kein temporäres Verzeichnis am Webserver, in das die Datei hochgeladen werden kann.

- `UPLOAD_ERR_CANT_WRITE` (7): Der Webserver kann die Datei nicht ins Dateisystem schreiben (z. B. wegen fehlender Schreibrechte).
- `UPLOAD_ERR_EXTENSION` (8): Eine PHP-Erweiterung hat einen Fehler verursacht.

Es existiert kein Fehlercode mit dem Wert 5 mehr. Dieser wurde ursprünglich erzeugt, wenn eine leere Datei hochgeladen wurde, dieses Verhalten wurde aber später nicht mehr als Fehler angesehen und der Code daher entfernt.

Bevor mit der weiteren Verarbeitung der hochgeladenen Datei fortgefahren wird, sollte der Fehlercode immer abgefragt werden. Lautet dieser 0, so war alles in Ordnung, ist ein anderer Wert im Array enthalten, so sollte der Fehlercode interpretiert werden. Die Fehler 1–4 können von den Nutzer*innen behoben werden (etwa durch erneutes Hochladen oder Hochladen einer kleineren Datei). Die Fehler 6–8 müssen von dem*der Betreiber*in des Webserver korrigiert werden. In diesem Fall bringt eine genaue Fehlermeldung an die Benutzer*innen nur bedingt etwas.

7.3.4 Verschieben der Datei und Sicherheitsaspekte

Nachdem sichergestellt wurde, dass der Upload funktioniert hat, muss die Datei aus dem temporären Verzeichnis an ihren Bestimmungsort verschoben werden. Das geschieht mit der Funktion `move_uploaded_file($filename, $destination)`⁶. Diese prüft zunächst, ob die angegebene Datei tatsächlich durch einen gültigen HTTP-POST-Upload entstanden ist, was das Einschleusen lokaler Systemdateien durch Angreifer verhindert. Danach verschiebt sie die Datei an das angegebene Ziel.

Sicherheitswarnung: Die Verwendung von `basename()` für den Dateinamen (wie im folgenden Beispiel) verhindert zwar sogenannte *Directory Traversal*-Angriffe (z. B. das Einschleusen von Pfaden wie `../../../../../etc/passwd`), löst aber nicht alle Sicherheitsprobleme. Wenn der Originalname beibehalten wird, können bestehende Dateien überschrieben werden oder ausführbare Skripte (z. B. `shell.php`) auf den Server gelangen. Für Produktionssysteme sollten Dateinamen serverseitig immer neu generiert (z. B. mit `uniqid()`) und die Dateiendung anhand des vom Server geprüften MIME-Typs vergeben werden.

Der folgende Code zeigt die Verarbeitung der hochgeladenen Datei, die Auswertung spezifischer Fehlercodes über ein Array und das Verschieben unter Verwendung von `basename()`, um Pfadmanipulationen durch den Browser zu unterbinden. Die Fehlermeldungen sind in einer eigenen Datei definiert und werden im Beispiel eingebunden:

```
1 $uploadErrors = [
2     UPLOAD_ERR_INI_SIZE    => "Die Datei überschreitet die in php.ini konfigurierte
    maximale Größe.",
3     UPLOAD_ERR_FORM_SIZE  => "Die Datei überschreitet die im Formular angegebene
    maximale Größe.",
4     UPLOAD_ERR_PARTIAL    => "Die Datei wurde nur teilweise hochgeladen.",
5     UPLOAD_ERR_NO_FILE    => "Es wurde keine Datei ausgewählt.",
6     UPLOAD_ERR_NO_TMP_DIR => "Temporäres Verzeichnis fehlt.",
7     UPLOAD_ERR_CANT_WRITE => "Datei konnte nicht auf dem Server gespeichert werden.",
8     UPLOAD_ERR_EXTENSION  => "Upload wurde durch eine PHP-Erweiterung abgebrochen.",
9 ];
```

⁶<https://www.php.net/manual/de/function.move-uploaded-file.php>

Dann erfolgt das eigentliche Upload-Handling:

```

1 require "upload_errors.php";
2
3 const UPLOAD_DIR = "uploads/";
4
5 if ($_SERVER["REQUEST_METHOD"] === "POST") {
6     $error = $_FILES["userfile"]["error"];
7
8     if ($error !== UPLOAD_ERR_OK) {
9         $message = $uploadErrors[$error] ?? "Unbekannter Fehler (Code $error).";
10        echo "<p>Fehler: $message</p>";
11    } else {
12        $originalName = basename($_FILES["userfile"]["name"]);
13
14        if (move_uploaded_file($_FILES["userfile"]["tmp_name"], UPLOAD_DIR .
            $originalName)) {
15            echo "<p>Upload von " . htmlspecialchars($originalName, ENT_QUOTES | ENT_HTML5
                ) . " erfolgreich!</p>";
16        } else {
17            echo "<p>Fehler: Datei konnte nicht gespeichert werden.</p>";
18        }
19    }
20 }
```

7.3.5 Hochladen mehrerer Dateien

Das Hochladen mehrerer Dateien gestaltet sich analog zu dem bisher gezeigten. Beim Formular sind zwei kleine Änderungen notwendig:

```

1 <form method="post" enctype="multipart/form-data"
    action="<?= $_SERVER["SCRIPT_NAME"] ?>">
2 <input type="hidden" name="MAX_FILE_SIZE" value="2097152">
3 <input type="file" name="userfiles[]" multiple>
4 <button type="submit">Hochladen</button>
5 </form>
```

Das Formularfeld trägt nun den Namen `userfiles[]`, repräsentiert also ein Array. Zudem wird das Attribut `multiple` hinzugefügt, um die Auswahl mehrerer Dateien gleichzeitig zuzulassen. Da es auf diese Weise im Browser nicht limitiert werden kann, wie viele Dateien maximal ausgewählt werden dürfen, können alternativ auch mehrere einzelne Eingabefelder mit demselben `name`-Attribut (jeweils mit eckigen Klammern) erstellt werden.

Die Daten landen wieder im `$_FILES`-Array. Dieses ist bei multiplen Uploads jedoch um eine Ebene tiefer verschachtelt. Jeder der regulären Einträge (wie `name`, `tmp_name` oder `error`) ist nun selbst ein indiziertes Array mit so vielen Einträgen, wie Dateien hochgeladen wurden.

Jede Datei wird einzeln verarbeitet. Dazu wird zunächst mittels `count()` bei einem der Einträge ermittelt, wie viele Dateien übermittelt wurden. Anschließend wird in einer Schleife über jeden Index iteriert. Dabei kommt dieselbe robuste Logik wie beim Einzel-Upload zum Einsatz: Der Originalname wird mittels `basename()` entschärft, die Ausgabe in HTML mit `htmlspecialchars()` maskiert und anhand des Fehlercodes entschieden, ob die Datei verschoben wird.

```

1 $nrOfFiles = count($_FILES["userfiles"]["name"]);
2 for ($i = 0; $i < $nrOfFiles; $i++) {
3     $error = $_FILES["userfiles"]["error"][$i];
4     $originalName = basename($_FILES["userfiles"]["name"][$i]);
5     $safeName = htmlspecialchars($originalName, ENT_QUOTES | ENT_HTML5);
6
7     if ($error !== UPLOAD_ERR_OK) {
8         $message = $uploadErrors[$error] ?? "Unbekannter Fehler (Code $error).";
9         echo "<p>Fehler bei $safeName: $message</p>";
10    } elseif (move_uploaded_file($_FILES["userfiles"]["tmp_name"][$i], UPLOAD_DIR .
        $originalName)) {
11        echo "<p>Upload von $safeName erfolgreich!</p>";
12    } else {
13        echo "<p>Fehler: $safeName konnte nicht gespeichert werden.</p>";
14    }
15 }

```

7.3.6 Hochladen ganzer Verzeichnisse

Seit PHP 8.1 gibt es die Möglichkeit, ein Verzeichnis auszuwählen und dessen gesamte Inhalte (inklusive aller Unterverzeichnisse) hochzuladen. Das Formular ähnelt dem für den Upload mehrerer Dateien, wird jedoch um das Attribut `webkitdirectory` ergänzt. Dieses Attribut ist formal nicht standardisiert, wird aber von allen modernen Browsern problemlos unterstützt.

```

1 <form method="post" enctype="multipart/form-data"
    action="<?= $_SERVER["SCRIPT_NAME"] ?>">
2 <input type="hidden" name="MAX_FILE_SIZE" value="2097152">
3 <input type="file" name="userfiles[]" webkitdirectory multiple>
4 <button type="submit">Hochladen</button>
5 </form>

```

Die Verarbeitung im `$_FILES`-Array verläuft ähnlich wie im vorigen Abschnitt. Um jedoch die Verzeichnisstruktur serverseitig identisch nachzubilden, muss anstelle des Schlüssels `name` der Schlüssel `full_path` ausgelesen werden. Dieser enthält neben dem Dateinamen auch den vom Browser übermittelten relativen Pfad.

Sicherheitswarnung: Wie der Dateiname wird auch der Wert `full_path` direkt vom Client übermittelt. Ein Angreifer kann diesen manipulieren und *Directory Traversal*-Vektoren wie `../` einschleusen. In einem Produktivsystem muss der Pfad vor der Weiterverarbeitung strikt validiert und bereinigt werden, um ein Ausbrechen aus dem Upload-Verzeichnis zu verhindern.

Bevor die Datei mit `move_uploaded_file()` an ihren Bestimmungsort verschoben wird, muss sichergestellt sein, dass das entsprechende Zielverzeichnis existiert. Der folgende Code extrahiert dazu mit `dirname()`⁷ den Ordnerpfad aus der Zielvariable und prüft mittels `is_dir()`⁸, ob dieser existiert. Ist das nicht der Fall, wird er mit `mkdir()`⁹ erstellt. Der dritte Parameter (`true`) sorgt dafür, dass das Anlegen rekursiv passiert,

⁷<https://www.php.net/manual/de/function.dirname.php>

⁸<https://www.php.net/manual/de/function.is-dir.php>

⁹<https://www.php.net/manual/de/function.mkdir.php>

also die gesamte Ordnerhierarchie aufgebaut wird. Die Dateiberechtigungen sind hier beispielhaft auf 0777 gesetzt, was ausschließlich in lokalen Entwicklungsumgebungen verwendet werden darf.

```

1 $nrOfFiles = count($_FILES["userfiles"]["name"]);
2 for ($i = 0; $i < $nrOfFiles; $i++) {
3     $error = $_FILES["userfiles"]["error"][$i];
4     $relativePath = $_FILES["userfiles"]["full_path"][$i];
5     $destination = UPLOAD_DIR . $relativePath;
6     $safePath = htmlspecialchars($relativePath, ENT_QUOTES | ENT_HTML5);
7
8     if ($error !== UPLOAD_ERR_OK) {
9         $message = $uploadErrors[$error] ?? "Unbekannter Fehler (Code $error).";
10        echo "<p>Fehler bei $safePath: $message</p>";
11    } elseif (!is_dir(dirname($destination)) && !mkdir(dirname($destination), 0777,
12        true)) {
13        echo "<p>Fehler: Verzeichnis für $safePath konnte nicht erstellt werden.</p>";
14    } elseif (move_uploaded_file($_FILES["userfiles"]["tmp_name"][$i], $destination))
15    {
16        echo "<p>Upload von $safePath erfolgreich!</p>";
17    } else {
18        echo "<p>Fehler: $safePath konnte nicht gespeichert werden.</p>";
19    }
20 }

```

7.3.7 bulletproof: Dateiuploads mit einer fertigen Komponente

Auch für Dateiuploads stehen auf Packagist fertige Komponenten zur Verfügung, die den Boilerplate-Code und die vielen manuellen Sicherheitschecks verringern. Für das sichere Hochladen von *Bildern* ist die leichtgewichtige Komponente **samayo/bulletproof** eine gute Wahl. Da sich Bulletproof ausschließlich auf Bildformate spezialisiert hat, sollten für den Upload beliebiger anderer Dateitypen (z.B. PDFs, Office-Dokumente oder ZIP-Archive) generische Bibliotheken wie **siriusphp/upload** oder die robuste **UploadedFile**-Klasse der **symfony/http-foundation** Komponente herangezogen werden.

Bulletproof wird regulär über Composer installiert:

```
1 $ composer require samayo/bulletproof
```

Die Bibliothek abstrahiert das mühsame Arbeiten mit dem `$_FILES`-Array. Im folgenden kompletten Beispiel wird ein HTML-Formular bereitgestellt. Sobald dieses per POST abgesendet wird, übernimmt Bulletproof die Validierung.

Zunächst wird ein neues **Image**-Objekt mit dem superglobalen Array instanziiert. Anschließend werden die Sicherheitsregeln als Kettenaufrufe (Method Chaining) konfiguriert. In diesem Beispiel wird der Upload auf gängige Bildformate bis maximal 2 Megabyte ($2 * 1024 * 1024$ Bytes) beschränkt und das Zielverzeichnis mit **setStorage()** festgelegt.

```

1 $image = new Bulletproof\Image($_FILES);
2 $image
3     ->setStorage("uploads/")
4     ->setMime(["jpeg", "png", "gif"])
5     ->setSize(1, 2 * 1024 * 1024);
6

```

```
7 // Prüfen, ob eine Datei im Upload-Feld übergeben wurde
8 if ($image["userfile"]) {
9     $upload = $image->upload();
10
11     if ($upload) {
12         // getName() liefert den von Bulletproof neu generierten, sicheren Namen
13         echo "<p>Upload von {$upload->getName()} erfolgreich!</p>";
14     } else {
15         echo "<p>Fehler: {$image->getError()}</p>";
16     }
17 } else {
18     echo "<p>Fehler: {$image->getError()}</p>";
19 }
```

Der finale Upload-Prozess wird durch den Aufruf von `upload()` angestoßen. Bulletproof kümmert sich dabei automatisch um die Vergabe eines sicheren, zufällig generierten Dateinamens. Zudem prüft die Bibliothek die echte Datei-Signatur (MIME-Type) des Inhalts, um sicherzustellen, dass sich nicht etwa ein ausführbares Skript hinter einer gefälschten `.jpg`-Endung versteckt. Liefert die Methode `true` (bzw. das generierte Image-Objekt) zurück, war der Upload erfolgreich; andernfalls kann die exakte Fehlerursache über `getError()` ausgelesen werden.

7.4 Weiterführende Literatur

Als primäre Referenz für Formularverarbeitung, Filter-Funktionen und Dateiuploads sollte in diesem Fall [2] eingesetzt werden. Sie ist naturgemäß am aktuellsten Stand, während die meisten Bücher zu dem Thema oft veraltete Informationen beinhalten. Etwa wenn es um diverse Parameter von Funktionen geht.

Dateiuploads werden ausführlich in [1, 3, 4] behandelt, jedoch decken die Bücher dabei nur unwesentlich mehr Details ab, als in dieser Vorlesung vorgestellt wurden.

Quellenverzeichnis

- [1] Kevin Tatroe und Peter MacIntyre. *Programming PHP. Creating Dynamic Web Pages*. 4. Aufl. Sebastopol, USA: O'Reilly, 31. März 2020 (siehe S. 7-19).
- [2] The PHP Documentation Group. *PHP-Handbuch*. 19. Mai 2026. URL: <https://www.php.net/manual/de/> (siehe S. 7-19).
- [3] Thomas Theis. *Einstieg in PHP 8 und MySQL*. 15. Aufl. Bonn, Deutschland: Rheinwerk Computing, 5. Apr. 2023 (siehe S. 7-19).
- [4] Christian Wenz und Tobias Hauser. *PHP 8 und MySQL. Das umfassende Handbuch*. 5. Aufl. Bonn, Deutschland: Rheinwerk Computing, 3. Mai 2024 (siehe S. 7-19).